INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	*Computación gráfica, visión computacional y multimedia*				
TÍTULO DE LA PRÁCTICA:	*Cámara primera y tercera persona*				
NÚMERO DE PRÁCTICA:	*04*	**AÑO LECTIVO:**	*2026-A*	**NRO. SEMESTRE:**	*VII*
FECHA DE PRESENTACIÓN	*10/05/2026*	**HORA DE PRESENTACIÓN**	*23:59 pm*		
INTEGRANTE (s): Perez Huamani Jeremy Joshua Soncco Mamani Max Junior				**NOTA:**	
DOCENTE(s): Prof. René Nieto					

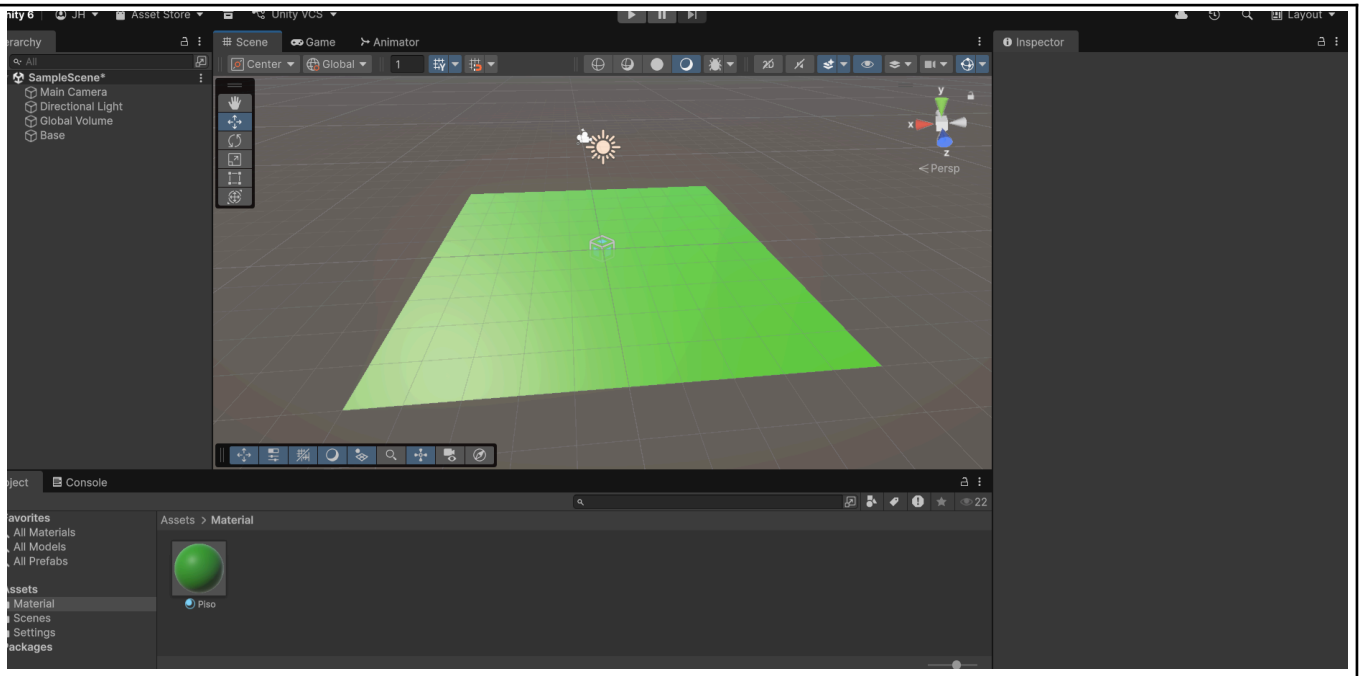
SOLUCIÓN Y RESULTADOS

I. EJERCICIOS PROPUESTOS

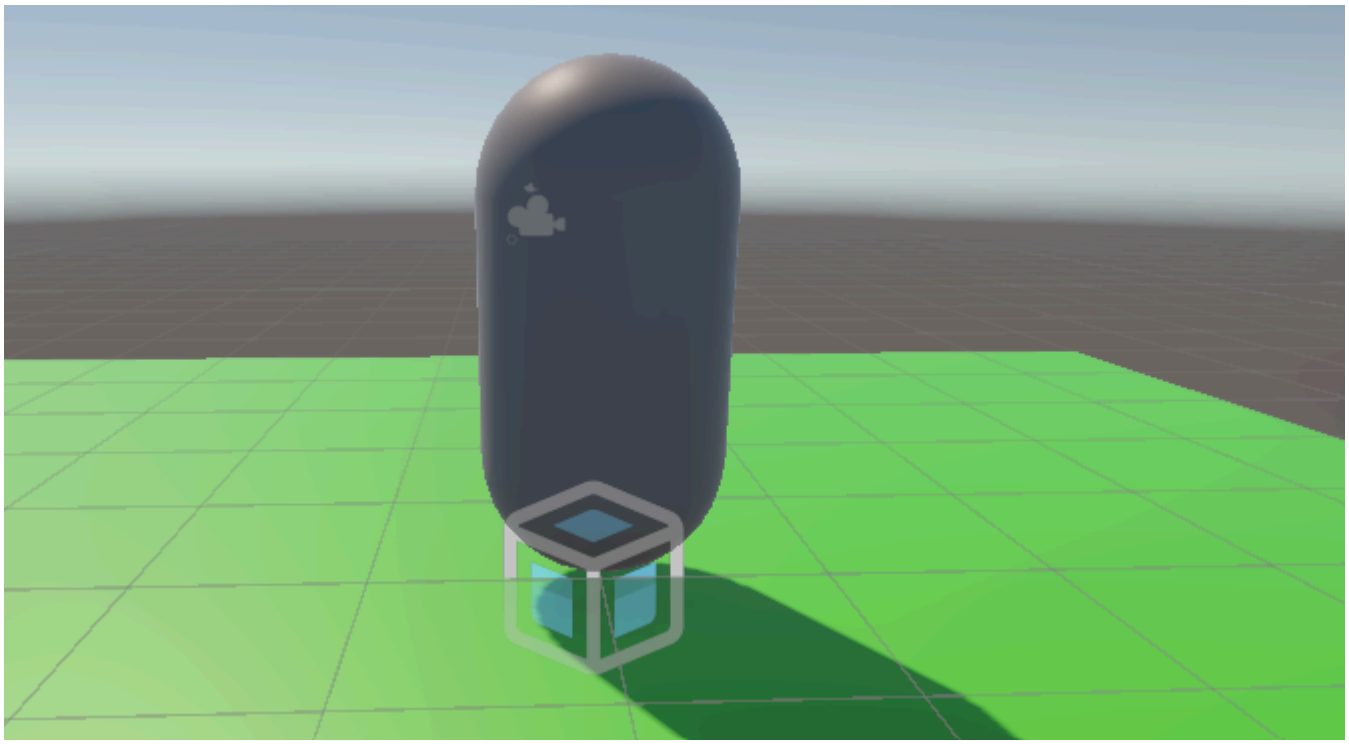
Carpeta con videos:

https://drive.google.com/drive/folders/145lXy7GcCjjkhQQFxo6mXWNVrOO4AO7F

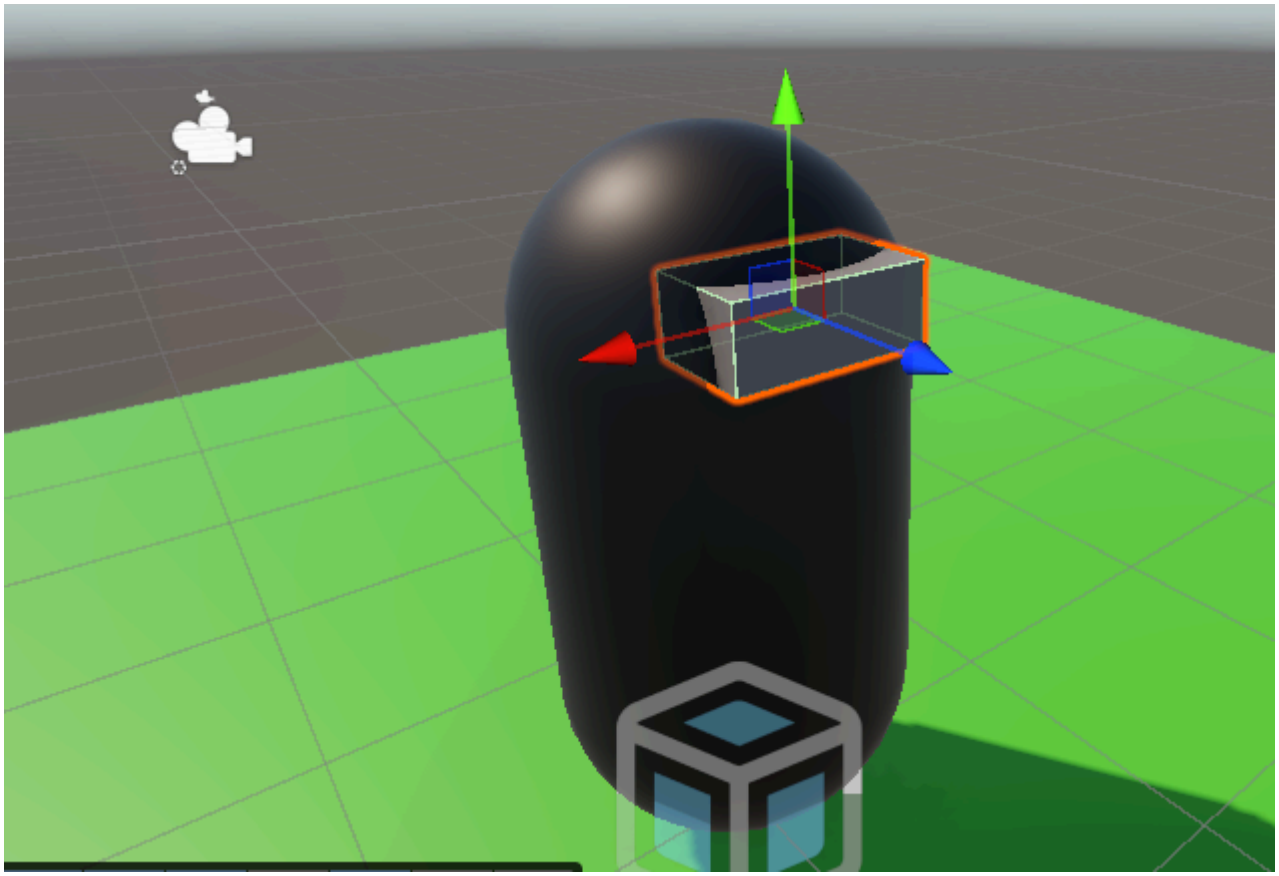
Empezamos creando el piso que será la plataforma donde se moverá nuestro personaje.



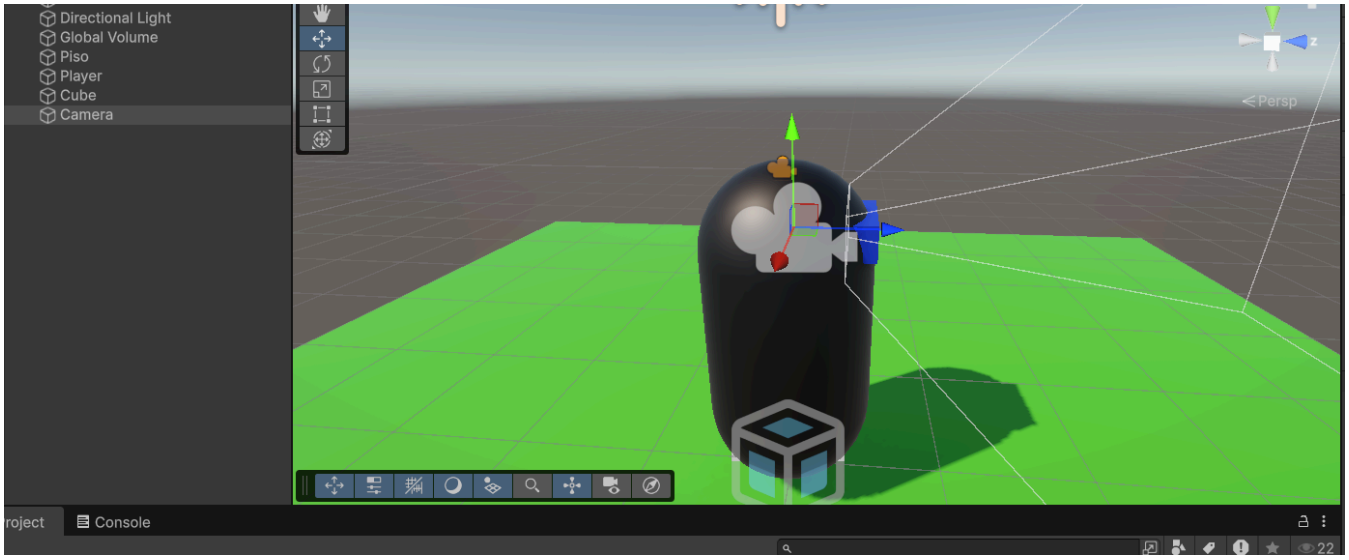
Esta tiene su propio material que será llamado piso.
Ahora creamos al personaje que utilizaremos que para este caso será una cápsula



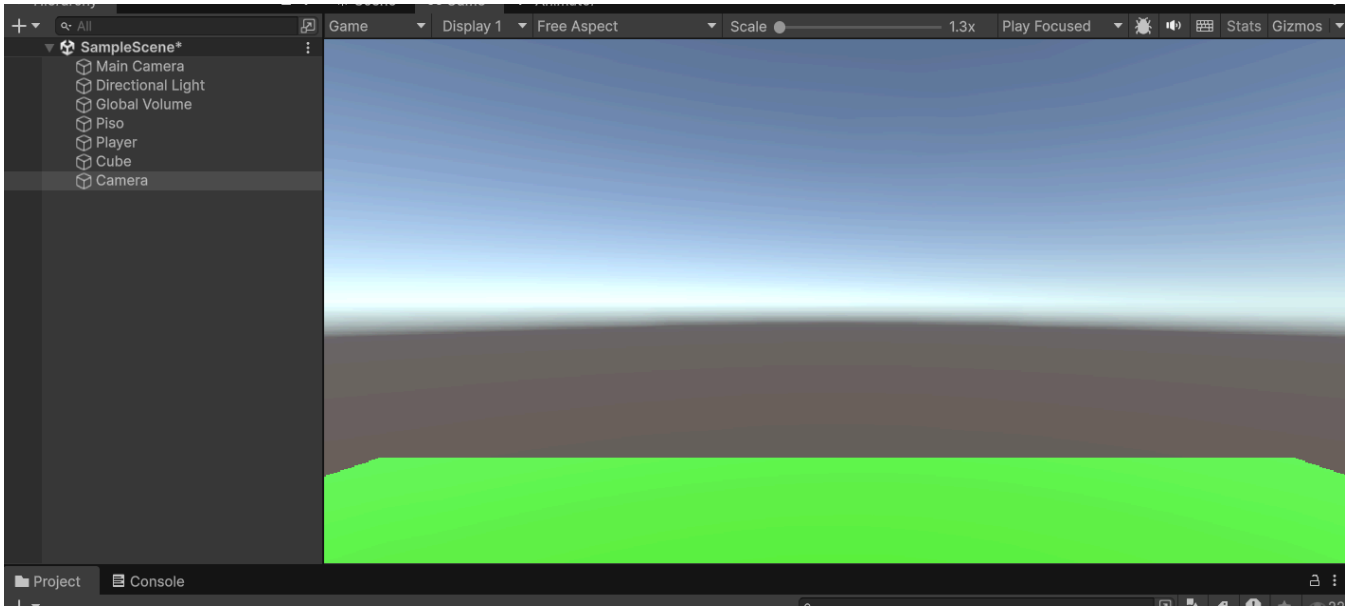
Creamos un cubo que será como el visor que tendrá nuestro personaje, necesitaremos eso para poder saber para dónde mira el personaje.



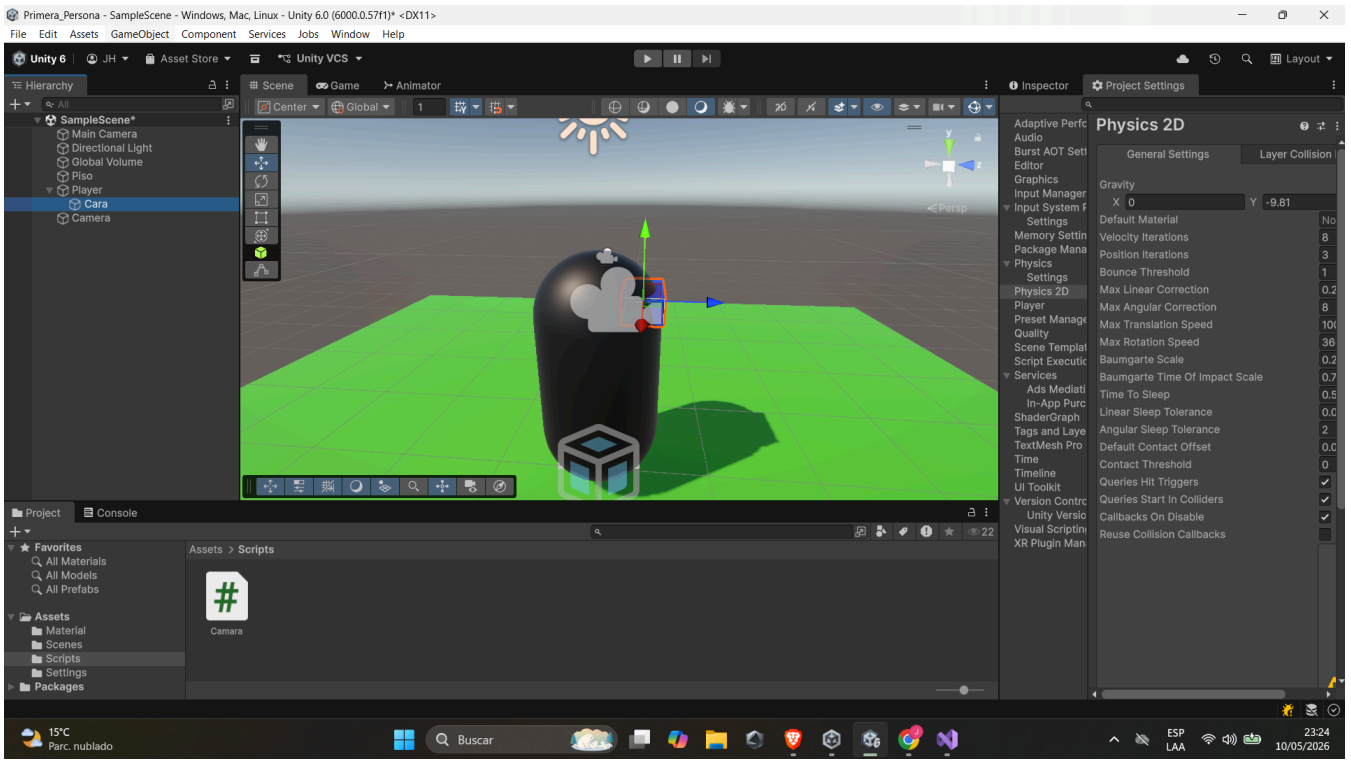
Ahora creamos una cámara a la altura de la cabeza del jugador.

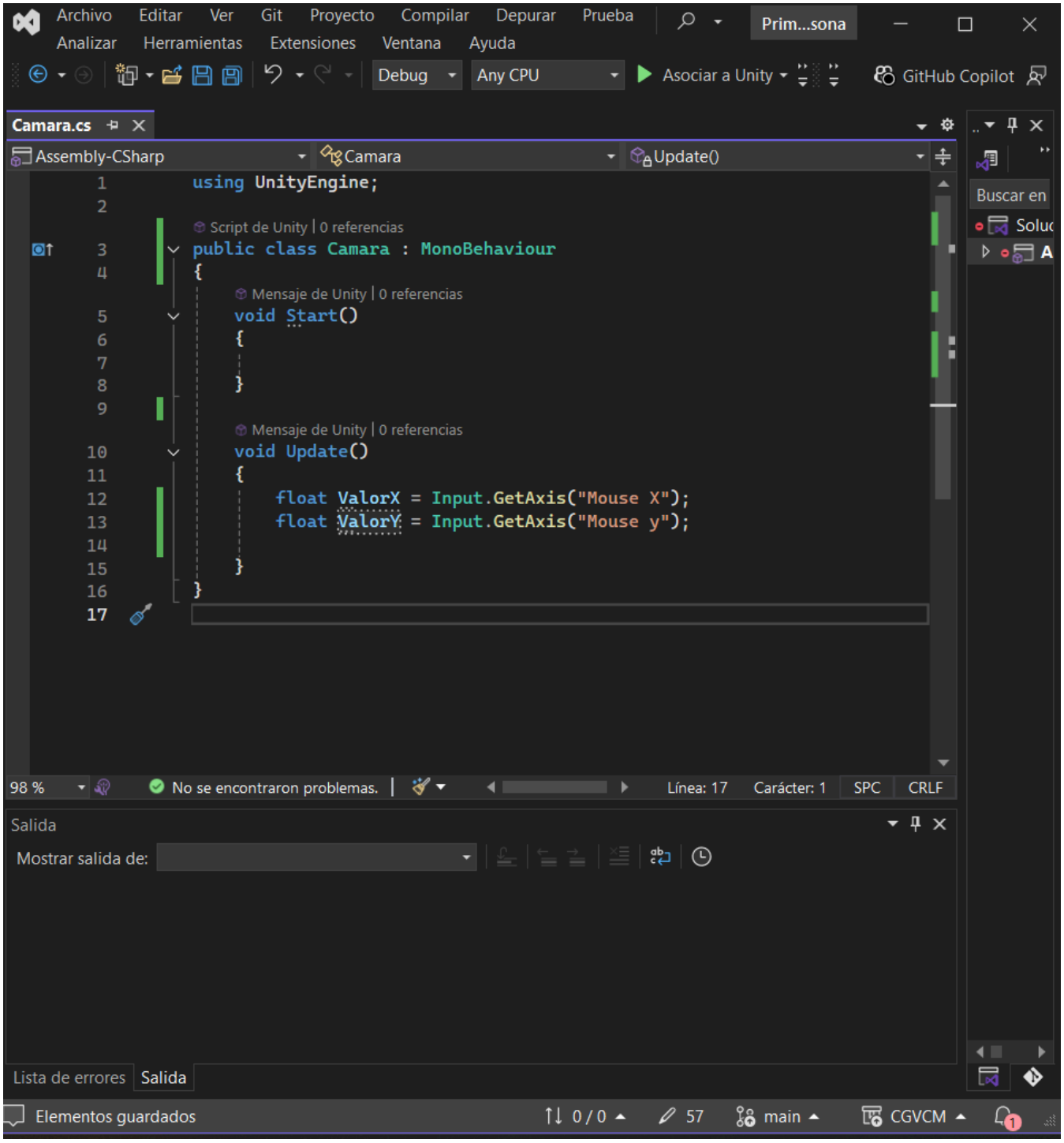


Como resultado en el juego se vería así.



esta ya es una vista en primera persona, claro que es necesario hacer que se pueda mover.
Ahora con esto hecho, vamos a empezar con los scripts, creamos su debida carpeta y empezamos con la captura de movimientos del mouse.





Esto captura el movimiento del mouse tanto en el eje x como el y.
Para entender mejor los movimiento que captura unity. añadimos esto.

Adaptive Performance
Audio
Burst AOT Settings
Editor
Graphics
Input Manager
Input System Package
Settings
Memory Settings
Package Manager
Physics
Settings
Physics 2D
Player
Preset Manager
Quality
Scene Template
Script Execution Order
Services
Ads Mediation (LevelPlay)
In-App Purchasing
ShaderGraph
Tags and Layers
TextMesh Pro
Time
Timeline
UI Toolkit
Version Control
Unity Version Control Settings
Visual Scripting
XR Plugin Management
Input Manager
Vertical
Fire1
Fire2
Fire3
Jump
Mouse X
Name
Mouse X
Descriptive Name
Descriptive Negative Name
Negative Button
Positive Button
Alt Negative Button
Alt Positive Button
Gravity
0
Dead
0
Sensitivity
0.1
Snap
Invert
Type
Mouse Movem
Axis
X axis
Joy Num
Get Motion fro
Mouse Y
Name
Mouse Y
Descriptive Name
Descriptive Negative Name
Negative Button
Positive Button
Alt Negative Button
Alt Positive Button
Gravity
0
Dead
0
Sensitivity
0.1
Snap

Para ver que pueden ser capturadas en nuestro unity, ese es el fin de añadirlo al lado del inspector.
Para manejar una velocidad constante de refresco, multiplicamos los valores por Time.deltaTime que hará que este valor sea constante.

```
t.GetAxis("Mouse X") * 100 * Time.deltaTime;
t.GetAxis("Mouse y") * 100 * Time.deltaTime;
```

Para manejarlo mejor, añadiremos la variable pública sensibilidad al código, para manejarlo desde el inspector.

```
Script de Unity | 0 referencias
public class Camara : MonoBehaviour
{

    public float Sensibilidad = 100;

    Mensaje de Unity | 0 referencias
    void Start()
    {

    }

    Mensaje de Unity | 0 referencias
    void Update()
    {
        float ValorX = Input.GetAxis("Mouse X") * Sensibilidad * Time.deltaTime;
        float ValorY = Input.GetAxis("Mouse y") * Sensibilidad* Time.deltaTime;

    }
}
```

Añadimos una sumatoria para manejar valores límites en eje x y y

Camara.cs

Assembly-CSharp | Camara | Update()

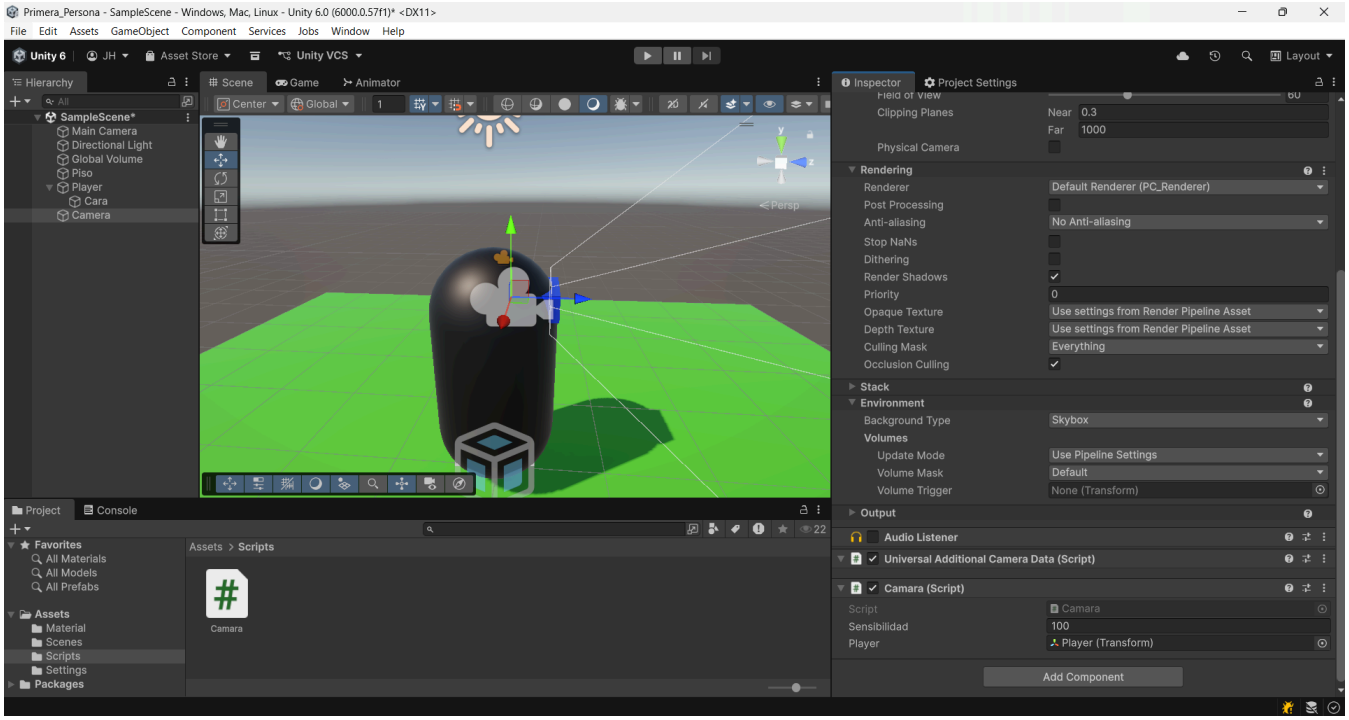
```
using UnityEngine;

Script de Unity | 0 referencias
public class Camara : MonoBehaviour
{

    public float Sensibilidad = 100;


    float RotacionHorizontal = 0;
    float RotacionVertical = 0;
    Mensaje de Unity | 0 referencias
    void Start()
    {

    }

    Mensaje de Unity | 0 referencias
    void Update()
    {
        float ValorX = Input.GetAxis("Mouse X") * Sensibilidad * Time.deltaTime;
        float ValorY = Input.GetAxis("Mouse y") * Sensibilidad * Time.deltaTime;

        RotacionHorizontal += ValorX;
        RotacionVertical += ValorY;

    }
}
```

Camara.cs

Assembly-CSharp | Camara | Update()

```
using UnityEngine;

Script de Unity | 0 referencias
public class Camara : MonoBehaviour
{

    [SerializeField] public float Sensibilidad = 100;

    public Transform Player;

    float RotacionHorizontal = 0;
    float RotacionVertical = 0;
    Mensaje de Unity | 0 referencias
    void Start()
    {

    }

    Mensaje de Unity | 0 referencias
    void Update()
    {
        float ValorX = Input.GetAxis("Mouse X") * Sensibilidad * Time.deltaTime;
        float ValorY = Input.GetAxis("Mouse y") * Sensibilidad * Time.deltaTime;

        RotacionHorizontal += ValorX;
        RotacionVertical += ValorY;

        transform.localRotation = Quaternion.Euler(RotacionVertical,0f,0f);

        Player.Rotate(Vector3.up * RotacionHorizontal);

    }
}
```

Para manejar el movimiento, al rotar en el eje x, se mueve todo el jugador, por ellos utilizamos Rotate y para el eje y, como no es necesario que se mueva del todo, utilizamos localRotation.
Con esto hecho, arrastramos player al inspector en cámara



Ahora limitamos el ángulo de giro vertical del personaje en 80 grados para que no sucedan casos como los del exorcista.

Camara.cs

Assembly-CSharp | CameraFollow | Update()

```
23          }
24
            Mensaje de Unity | 0 referencias
25          void Update()
26          {
27              //Nos dan los valores del mause para mover
28              float ValorX = Input.GetAxis("Mouse X") * Sensibilidad * Time.deltaTime;
29              float ValorY = Input.GetAxis("Mouse Y") * Sensibilidad * Time.deltaTime;
30
31              //Guarda el valor y queda en el valor para seguir
32              RotacionHorizontal += ValorX;
33              RotacionVertical -= ValorY;
34
35              //Limita en 80 grados
36              RotacionVertical = math.clamp(RotacionVertical, -80, 80);
37
38
39              //Hace la rotaion vertical fluida
40              transform.localRotation = Quaternion.Euler(RotacionVertical, 0, 0);
41
42
43              //Hace la rotacion horizontal
44              if (Player != null)
45              {
46                  Player.Rotate(Vector3.up * ValorX);
47              }
48              else
49              {
50                  //Si no tiene asignado el player avisas en consola
51                  Debug.LogWarning("Camara: asigna (Player) en el Inspector.");
52              }
53          }
54      }
```

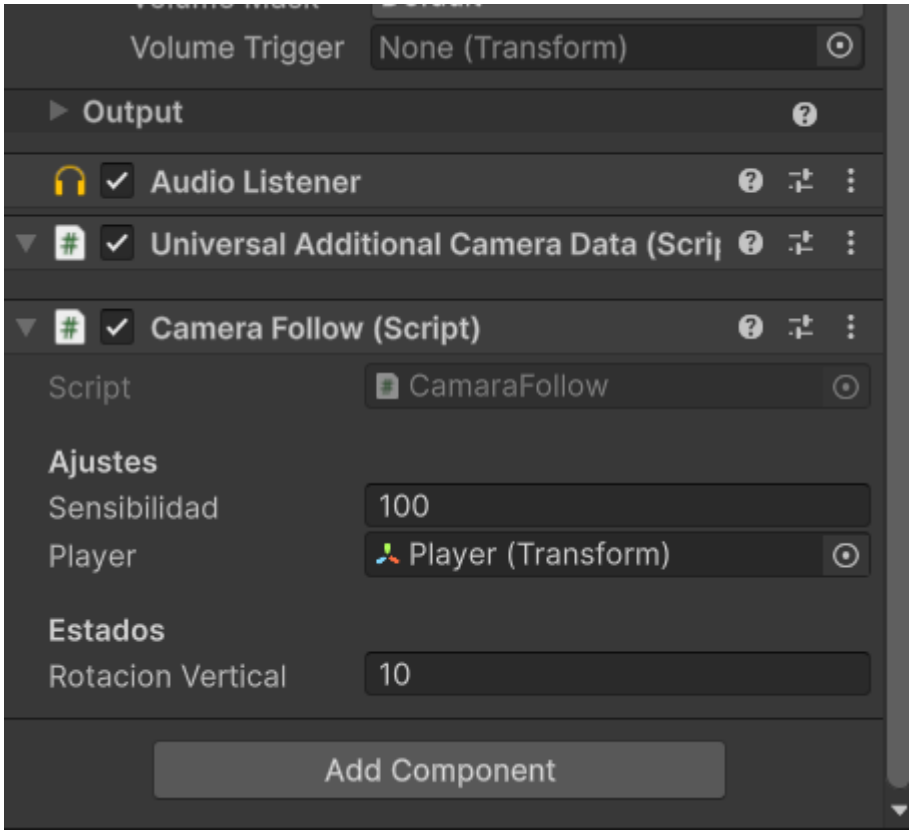
74 % | No se encontraron problemas. | Línea: 36 | Carácter: 13 | SPC | CRLF

Con esto bastaría para poder ver la cámara en primera persona del personaje.
Para hacer el movimiento del personaje. Colocamos el siguiente script.
Como ya se vio en anteriores laboratorios, capturamos el movimiento con wasd y lo limitamos con valores de velocidad.

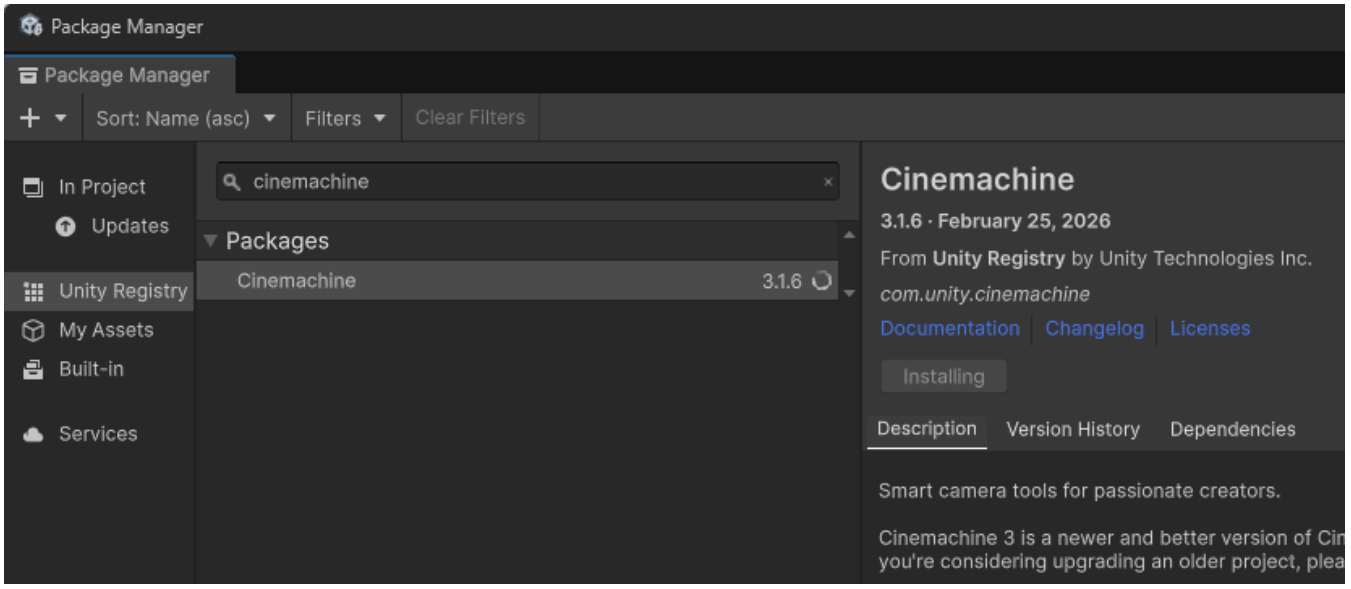
```
Script de Unity (1 referencia de recurso) | 0 referencias
public class PlayerMovement : MonoBehaviour
{
    [Header("Movimiento")]
    [SerializeField] private float velocidad = 5f;
    [SerializeField] private float velocidadCorrer = 8f;

    [Header("Salto y gravedad")]
    [SerializeField] private float fuerzaSalto = 1.5f;
    [SerializeField] private float gravedad = -9.81f;

    private CharacterController controller;
    private Vector3 velocidadVertical;

    Mensaje de Unity | 0 referencias
    void Start()
    {
        controller = GetComponent<CharacterController>();

        if (controller == null)
        {
            Debug.LogError("PlayerMovement: falta Character Controller en el Player.
```

```
Debug | Any CPU | Asociar a Unity
maraFollow.cs | PlayerMovement.cs | Camara.cs
Assembly-CSharp | PlayerMovement | Update()
29
30          float x = 0f;
31          float z = 0f;
32
33          if (Input.GetKey(KeyCode.A)) x = -1f;
34          if (Input.GetKey(KeyCode.D)) x = 1f;
35          if (Input.GetKey(KeyCode.W)) z = 1f;
36          if (Input.GetKey(KeyCode.S)) z = -1f;
37
38          Vector3 movimiento = transform.right * x + transform.forward * z;
39          movimiento = movimiento.normalized;
40
41          float velocidadActual = Input.GetKey(KeyCode.LeftShift) ? velocidadCorrer : velocidad;
42
43          controller.Move(movimiento * velocidadActual * Time.deltaTime);
44
45          if (controller.isGrounded && velocidadVertical.y < 0)
46          {
47              velocidadVertical.y = -2f;
48          }
49
50          if (Input.GetKeyDown(KeyCode.Space) && controller.isGrounded)
51          {
52              velocidadVertical.y = Mathf.Sqrt(fuerzaSalto * -2f * gravedad);
53          }
54
55          velocidadVertical.y += gravedad * Time.deltaTime;
56          controller.Move(velocidadVertical * Time.deltaTime);
57      }
58  }
% | No se encontraron problemas. | Línea: 39 | Carácter: 44 | SPC
```

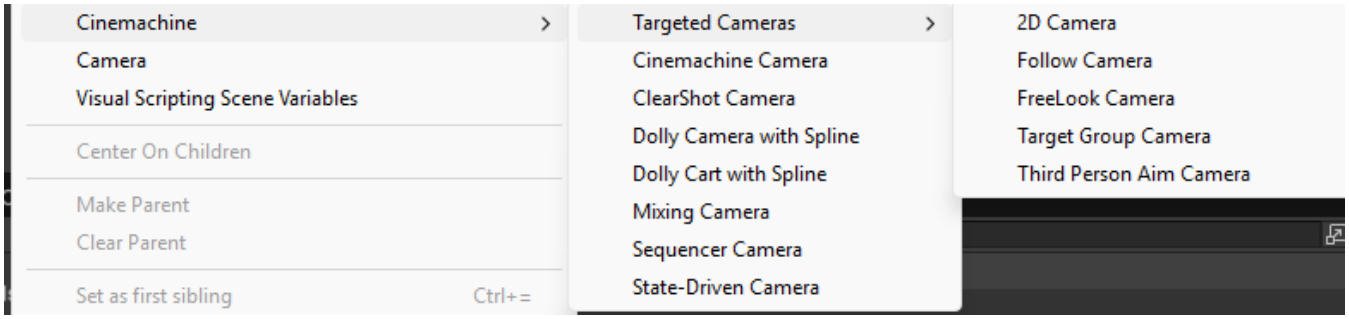
En la jerarquía añadimos el player al script de camera follow



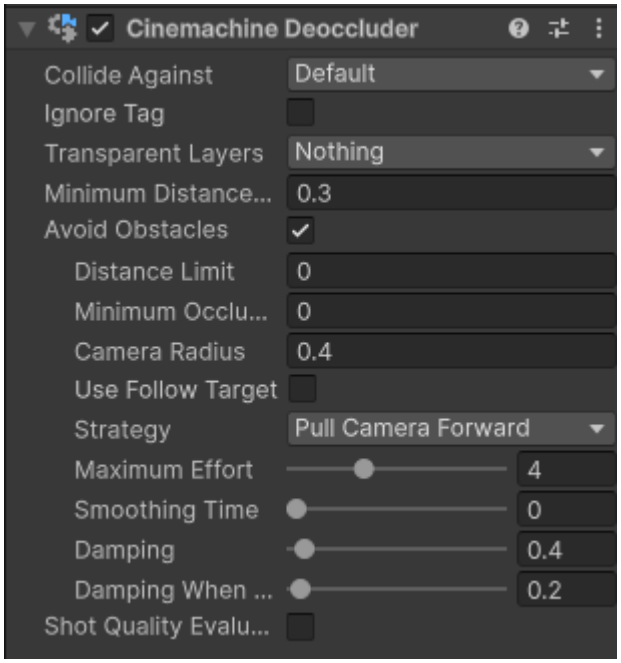
Con esto logramos el movimiento y visión en primera persona del jugador.
Para empezar con el desarrollo de una cámara en 3ra persona para el personaje, instalamos Cinemachine desde el Package Manager



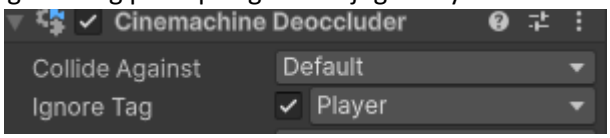
Generamos un objeto Cinemachine de tipo Follow Camera



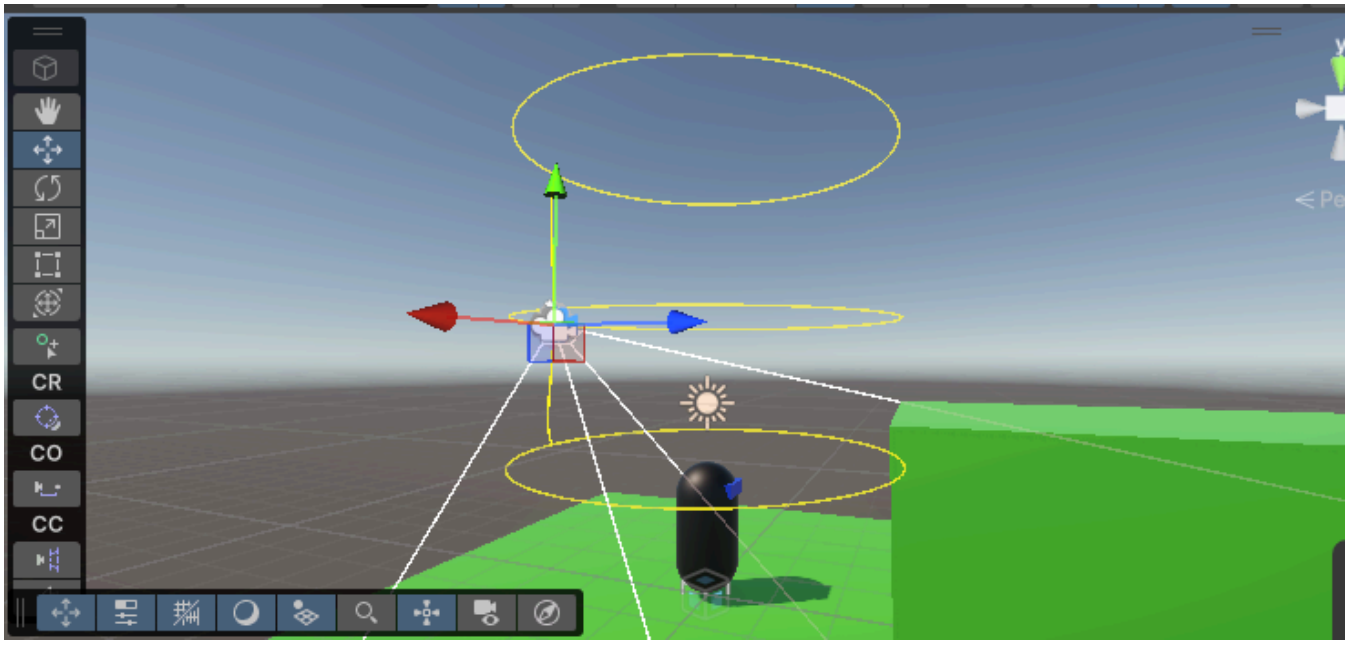
Al seleccionar el objeto, en Inspeccionar, agregamos un componente de tipo Collider, que para este caso se denomina Deoccluder, esto con el fin de que la cámara detecte colisiones y permita cambiar casi de manera automática



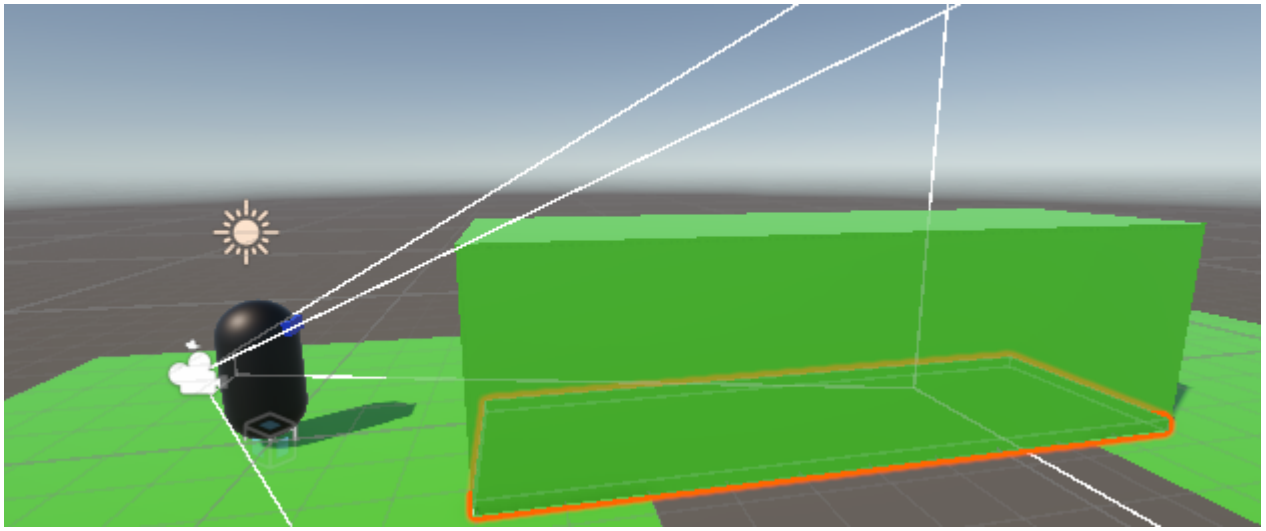
Configuramos la cámara principal para una vista en tercera persona, en el componente anterior le damos a Ignore Tag para que ignore al jugador y no lo tome como objeto de colision



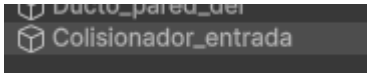
Además en el componente de seguimiento seleccionamos el estilo de órbita a 3 anillos y le damos radios iguales en los 3 niveles, esto con el fin de que la cámara no esté pegada a la espalda del Player



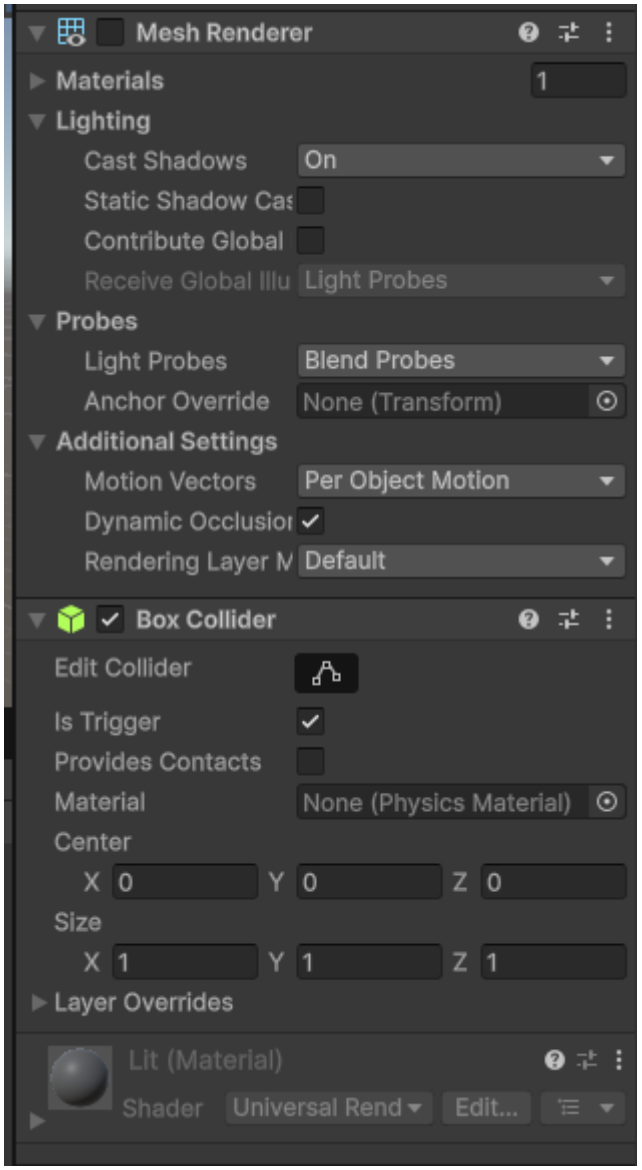
Para probar la jugabilidad y las escenas que vamos a incluir creamos un ducto por el cual el Player tendrá que pasar para llegar desde una plataforma A a una plataforma B



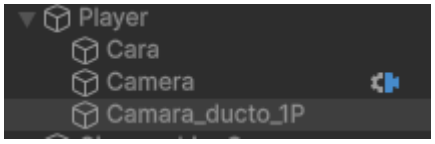
Luego de crear el ducto creamos un objeto para que funja como el colisionador de entrada al ducto, y que dispare el evento de cambio de camara



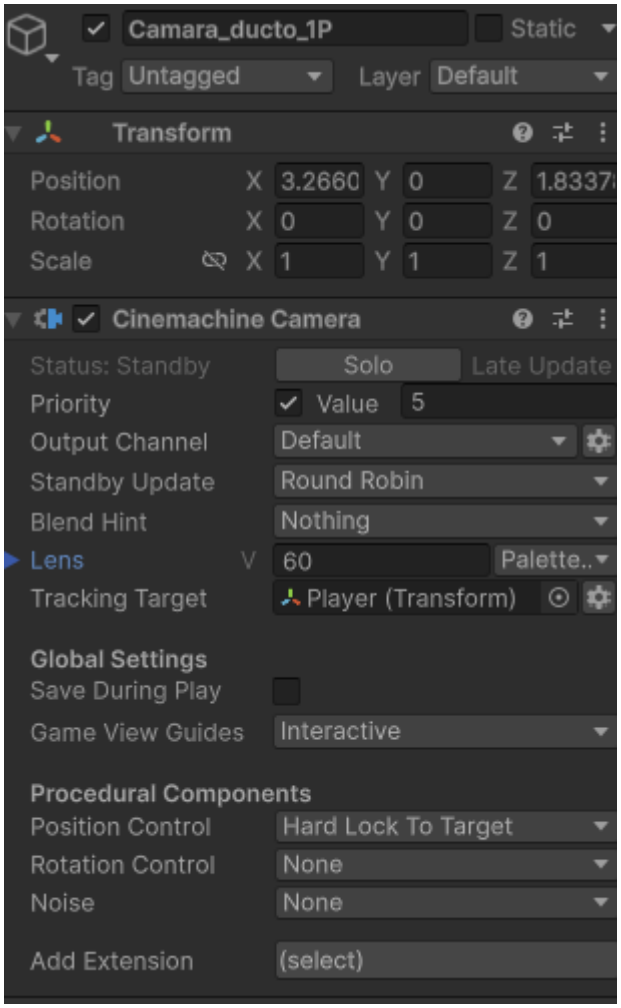
Deselccionamos el Mesh Renderer para hacerlo invisible y que el Player pueda pasar a través de él y en el componente Box Collider seleccionamos la casilla de Trigger para usarlo de flag para el evento mencionado anteriormente



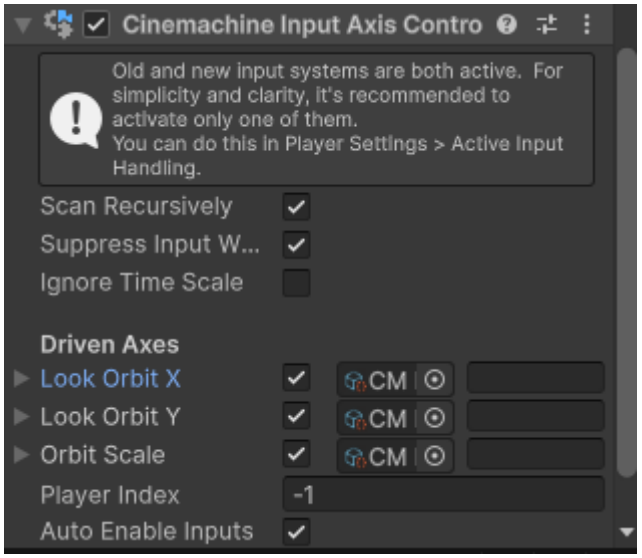
Creamos una segunda cámara, a la cuá nombramos Camara_ducto_1P, y lo ponemos dentro de la jerarquía de Player, esta va a fungir como la 2da cámara, y con la cual se va a realizar el cambio dentro del ducto



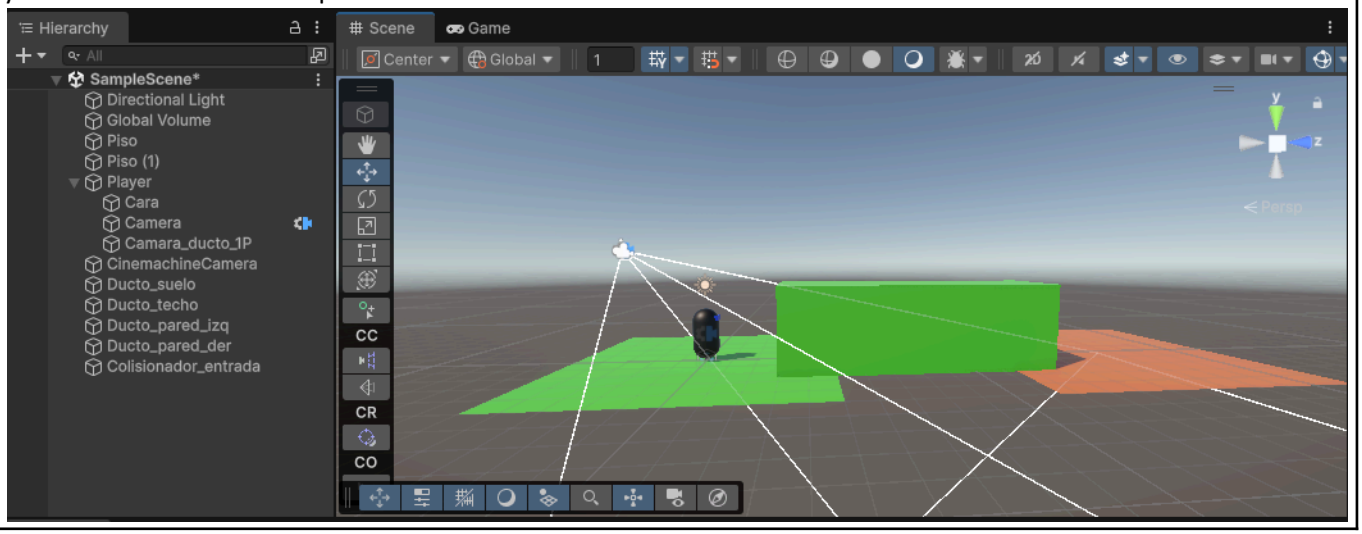
En el inspector definimos los valores de rotación en 0 en los tres componentes para que apunte adelante del jugador siempre

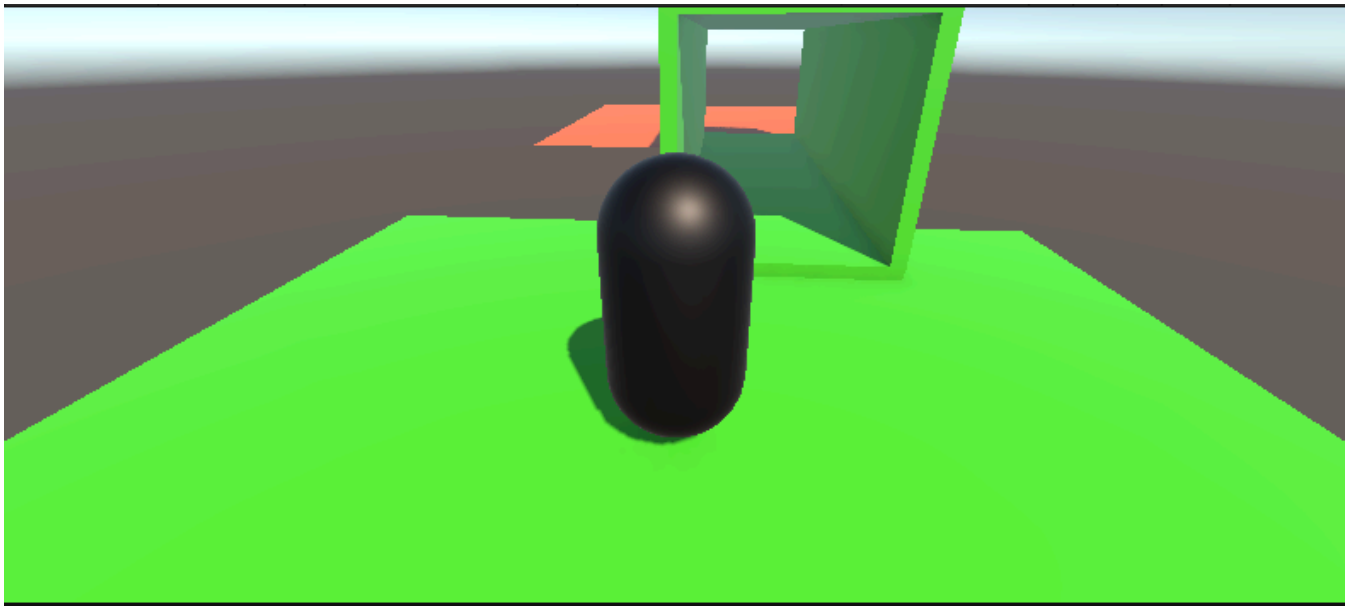


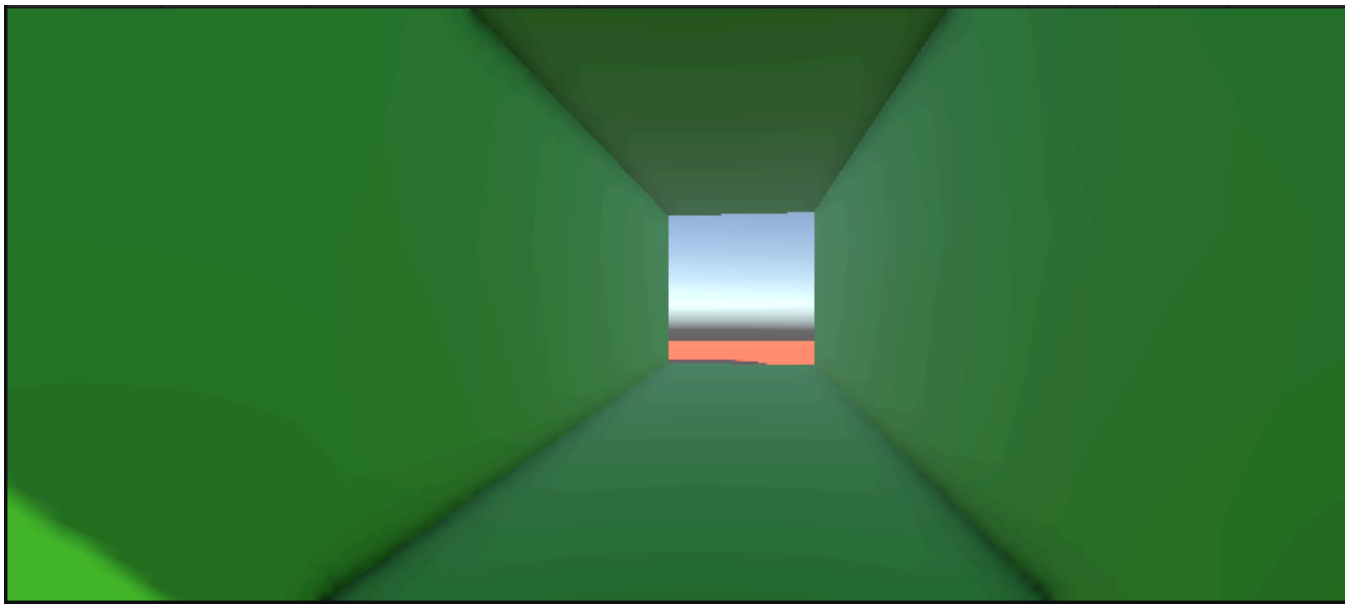
Para evitar que en la cámara de 3ra persona el jugador sea quien se mueva en respuesta al puntero añadimos el componente Input Axis Control, el cual permitirá que sea la cámara y el jugador quien responda al movimiento del puntero

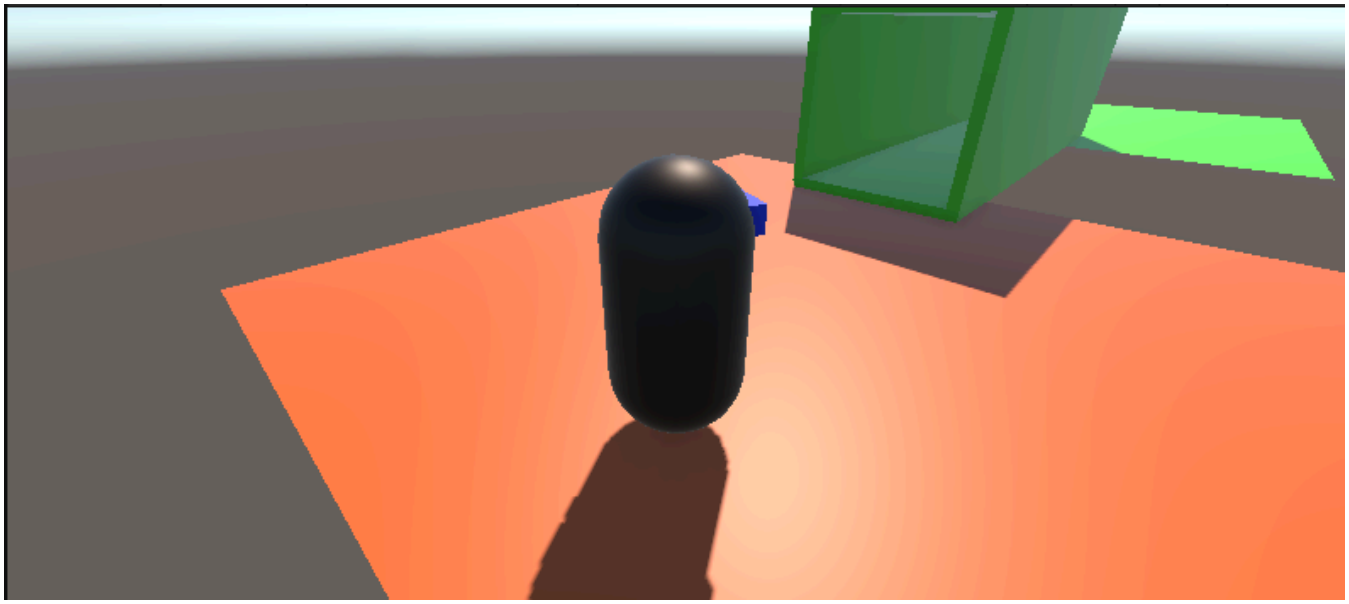


El resultado es el siguiente, tenemos una vista principal en 3ra persona, al pasar por el ducto está en 1ra persona y al salir del mismo en 3ra persona









II. SOLUCIÓN DEL CUESTIONARIO

No hay un cuestionario

III. CONCLUSIONES

A partir de la implementación realizada, se logró configurar un sistema básico de movimiento y cámara para el personaje dentro de Unity, separando correctamente las responsabilidades entre dos scripts. Por un lado, el script de cámara permite controlar la vista del jugador mediante el movimiento del mouse, diferenciando la rotación vertical de la cámara y la rotación horizontal del cuerpo del personaje. Por otro lado, el script de movimiento permite desplazar al jugador usando las teclas principales, tomando como referencia la dirección hacia donde este está mirando.

También se pudo evidenciar la importancia de organizar correctamente los componentes dentro de la jerarquía de Unity, ya que el funcionamiento depende bastante de que cada elemento esté en el objeto adecuado. En este caso, el PlayerMovement y el Character Controller deben estar en el objeto Player, mientras que el CameraFollow debe estar en la cámara hija del jugador. Si alguno de estos componentes se coloca en otro objeto, el sistema puede no responder correctamente o generar errores en la consola.

Además, se comprendió que el Character Controller es necesario para mover al personaje de forma controlada, ya que permite manejar desplazamiento, colisiones, gravedad y salto sin depender directamente de un Rigidbody. Esto facilita bastante el desarrollo de un controlador tipo primera persona, porque el movimiento se vuelve más estable y fácil de ajustar.

Finalmente, esta implementación permitió entender mejor cómo se relacionan la cámara, el jugador y los controles de entrada. Aunque al inicio surgieron errores, como la falta del

Character Controller o problemas con la rotación, estos ayudaron a identificar que en Unity no solo importa el código, sino también la correcta asignación de componentes en el Inspector. Por ello, se puede concluir que el sistema implementado sirve como una base funcional para seguir agregando mejoras, como animaciones, sonidos, detección de suelo, interacción con objetos o una cámara más pulida.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[]https://www.youtube.com/watch?v=GbmRt0wydQU
[]https://didigameboy.itch.io/jambo-jungle-free-sprites-asset-pack/download/eyJleHBpcmVzIjoxNzc2ODI3NDAxLCJpZCI6MTUxNzE0fQ%3d%3d%2eZOk0YHJ7il1eY2vFT9pM9e1%2fh1M%3d